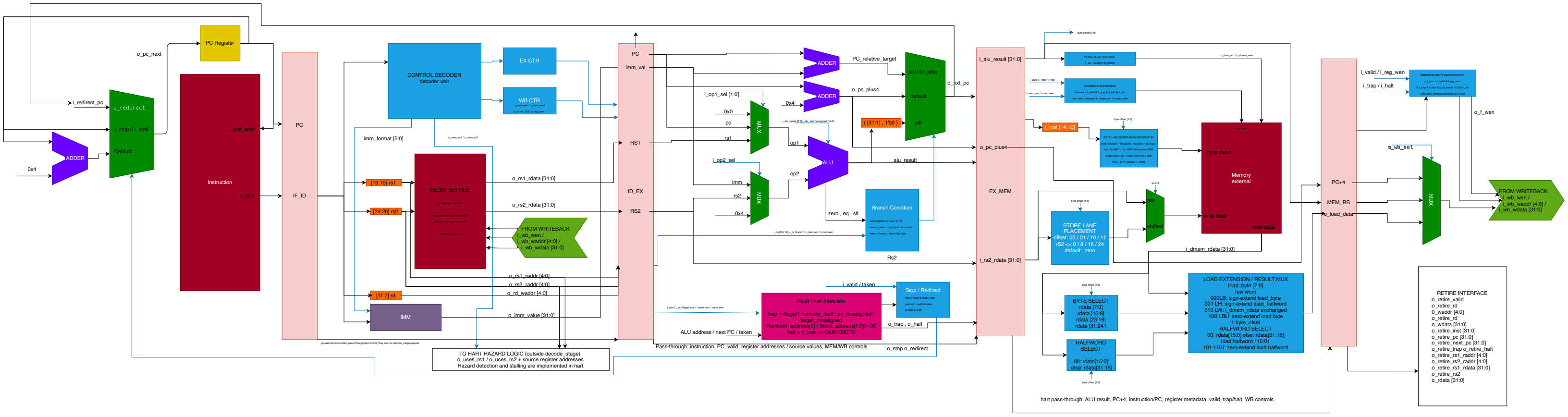
PC Register
o_pc_next
i_redirect_pc
i_redirect
i_stop || i_stall
Default
ADDER
0x4
Instruction
i_inst_addr
o_isnt
PC
IF_ID
CONTROL DECODER
decoder unit
EX CTR
WB CTR
o_mem_ren / o_mem_wen
o_b_sel [1:0] / o_reg_wen
imm_format [5:0]
o_uses_rs1 / o_uses_rs2
[19:15] rs1
[24:20] rs2
[11:7] rd
REGISTER FILE
32 registers x 32 bits
Two asynchronous read ports
One synchronous write port
BYPASS_EN = 0
FROM WRITEBACK
i_wb_wen /
i_wb_waddr [4:0] /
i_wb_wdata [31:0]
o_rs1_rdata [31:0]
o_rs2_rdata [31:0]
o_rs1_raddr [4:0]
o_rs2_raddr [4:0]
o_rd_waddr [4:0]
IMM
o_imm_value [31:0]
pcvalid and instruction pass through hart to D/X; they are not decode_stage outputs
TO HART HAZARD LOGIC (outside decode_stage)
o_uses_rs1 / o_uses_rs2 + source register addresses
Hazard detection and stalling are implemented in hart
PC
imm_val
RS1
ID_EX
RS2
i_op1_sel [1:0]
0x0
pc
rs1
MUX
i_op2_sel
imm
rs2
0x4
MUX
op1
op2
i_alu_opsel [2:0] i_alu_sub / unsigned / arith
ALU
alu_result
zero , eq , slt
ADDER
PC_relative_target
ADDER
0x4
o_pc_plus4
{ [31:1] , 1'b0 }
jal || br_take
default
jalr
o_nxt_pc
Branch Condition
funct selects eq / leq / lt / ltt
branch_taken = is_branch & condition
taken = branch_taken | jal | jalr
i_inst[14:12]+i_is_branch / i_men_ren / i_memwen
Rs2
i_inst / i_pc /illegal_inst / i mem ren / i mem wen
Fault / halt detection
trap = illegal | memory_fault | pc_misaligned | target_misaligned
Halfword: address[0] | Word: address[1:0]!= 00
halt = (i_inst == 0x00100073)
ALU address / next PC / taken
i_valid / taken
Stop / Redirect
stop = valid & (trap | halt)
redirect = valid & taken
& !trap & !halt
o_trap , o_halt
o_stop o_redirect
Pass-through: instruction, PC, valid, register addresses / source values, MEM/WB controls
i_alu_result [31:0]
o_pc_plus4
EX_MEM
i_rs2_rdata [31:0]
byte offset [1:0]
WORD-ALIGN ADDRESS
{i_alu_result[31:2], 2'b00}
o_dem_ren / o_dmem_wen
i_valid / i_trap / i_halt
i mem_ren / i mem wen
ACCESS QUALIFICATION
allowed = i_valid & !i_trap & !i_halt & !i_rst
ren / wen = allowed & i_mem_ren / i_mem_wen
i_inst [14:12]
byte offset [1:0]
BYTE / HALFWORD MASK GENERATOR
byte: 00=0001 / 01=0010 / 10=0100 / 11=1000
half: 00=0011 / 10=1100 / otherwise=0000
funct3: 000/100 -> byte / 001/101 > half
010 > 1111 / default -> 0000
ren / wen
byte mask
Memory
external
write data
read data
func 3
raw
shifted
byte offset [1:0]
STORE LANE PLACEMENT
offset: 00 / 01 / 10 / 11
rS2 << 0 / 8 / 16 / 24
default: zero
i_dmem_rdata [31:0]
byte offset [1:0]
BYTE SELECT
rdata [7:0]
rdata [15:8]
rdata [23:16]
rdata [31:241
HALFWORD SELECT
00: rdata[15:0]
else: rdata[31:16]
byte offset [1:0]
LOAD EXTENSION / RESULT MUX
load_byte [7:0]
raw word
000LB: sign-extend load_byte
001 LH: sign-extend load_halfword
010 LW: i_dmem_rdata unchanged
100 LBU: zero-extend load byte
1 byte_ofset
HALFWORD SELECT
00: rdata[15:0] else: rdata[31:16]
load halfword 115:01
101 LHU: zero-extend load halfword
hart pass-through: ALU result, PC+4, instruction/PC, register metadata, valid, trap/halt, WB controls
PC+4
MEM_RB
o_load_data
i_valid / i_reg_wen
i_trap / i_halt
REGISTER-WRITE QUALIFICATION
o_f_wen = i_valid & i_reg_wen
& !i_trap & !i_halt & (i_rd_waddr != 0) & !i_rst
Only valid, nonfaulting writes to x1-x31
o_f_wen
m_wb_sel
MUX
FROM WRITEBACK
i_wb_wen /
i_wb_waddr [4:0] /
i_wb_wdata [31:0]
RETIRE INTERFACE
o_retire_valid
o_retire_rd
0_waddr [4:0]
o_retire_rd
o_wdata [31:0]
o_retire_inst [31:0]
o_retire_pc [31:0]
o_retire_next_pc [31:0]
o_retire_trap o_retire_halt
o_retire_rs1_raddr [4:0]
o_retire_rs2_raddr [4:0]
o_retire_rs1_rdata [31:0]
o_retire_rs2
o_rdata [31:0]